

Computer Networks



Overview of the Internet Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
 - World Wide Web and HTTP
 - FTP
 - Electronic Mail
 - DNS
- Peer-to-peer paradigm
 - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

① Introduction

- Providing Services
- Application-Layer Paradigms

② Standard Client-Server Applications

- World Wide Web and HTTP
- FTP
- Electronic Mail
- Domain Name System (DNS)

③ Peer-to-Peer Paradigm

- P2P Networks

HyperText Transfer Protocol (HTTP)

- HTTP stands for:
 - HyperText Transfer Protocol
- HTTP defines:
 - Communication between web client and web server
- Used for:
 - Retrieving web pages from the Web
- HTTP client:
 - Sends request
- HTTP server:
 - Returns response

HTTP Communication

- Server uses:
 - Port number 80
- Client uses:
 - Temporary port number
- HTTP uses services of:
 - TCP
- TCP provides:
 - Connection-oriented communication
 - Reliable communication

Role of TCP in HTTP

- Before HTTP transaction:
 - TCP connection established
- After transaction:
 - TCP connection terminated
- TCP handles:
 - Error control
 - Message loss recovery
- Client and server do not need to:
 - Manage transmission errors

HTTP and Multiple Objects

- Web pages may contain:
 - Multiple linked objects
- Objects may be stored:
 - On same server
 - On different servers
- Different servers require:
 - Separate TCP connections
- Same server offers:
 - Two connection strategies

Nonpersistent vs Persistent Connections

- Two HTTP connection strategies:
 - Nonpersistent connections
 - Persistent connections
- Nonpersistent:
 - One TCP connection per request/response
- Persistent:
 - One TCP connection for multiple objects
- HTTP versions:
 - Before HTTP 1.1 → Nonpersistent
 - HTTP 1.1 → Persistent by default

Nonpersistent Connections

- One TCP connection used for:
 - One request
 - One response
- Connection closed after:
 - Response transmission
- New request requires:
 - New TCP connection
- Suitable for:
 - Simple communication

Steps in Nonpersistent Connections

- ❶ Client opens TCP connection and sends request
 - ❷ Server sends response and closes connection
 - ❸ Client reads data until end-of-file marker
 - ❹ Client closes connection
- Each request-response pair:
 - Uses separate connection

Overhead in Nonpersistent Connections

- Suppose a file contains:
 - N linked images
- Total TCP connections required:
 - $N + 1$
- Reason:
 - One connection for main file
 - N connections for images
- Nonpersistent strategy causes:
 - High server overhead
- Server requires:
 - Separate buffer for each connection

Comparison of HTTP Connection Types

Feature	Nonpersistent	Persistent
TCP Connections	Multiple	Single
Connection Reuse	No	Yes
Overhead	High	Lower
Efficiency	Lower	Higher
HTTP Version	Before 1.1	HTTP 1.1 Default

Example 2.4: Nonpersistent Connection

- Client accesses:
 - Text file containing one image link
- Text file and image:
 - Stored on same server
- Nonpersistent HTTP requires:
 - Separate TCP connection for each object
- Total connections required:
 - Two connections

Connection Establishment in HTTP

- Each TCP connection requires:
 - Three-way handshake
- At least three messages needed:
 - To establish connection
- Request can be sent:
 - Along with third handshake message
- After connection establishment:
 - Object transferred

Connection Termination

- After object transfer:
 - TCP connection terminated
- TCP termination requires:
 - Three handshake messages
- Each object retrieval causes:
 - One connection establishment
 - One connection termination

Total Operations in Example 2.4

- Two objects retrieved:
 - Main text file
 - Image file
- Total operations:
 - Two TCP connection establishments
 - Two TCP connection terminations
- Each connection introduces:
 - Additional delay
 - Additional overhead

Overhead of Nonpersistent Connections

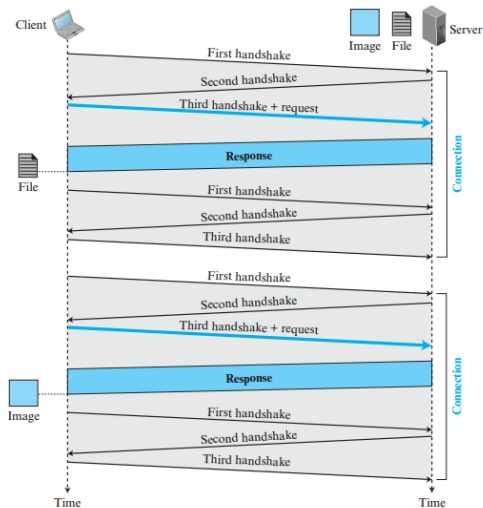
- Retrieving many objects increases:
 - Round-trip times
- Example:
 - 10 or 20 objects require many handshakes
- Handshake overhead becomes:
 - Significant
- Nonpersistent connections reduce:
 - Communication efficiency

Resource Usage in Nonpersistent Connections

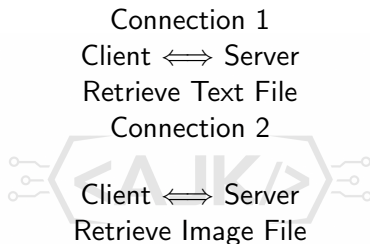
- Each TCP connection requires:
 - Buffers
 - Variables
 - System resources
- Resource allocation needed at:
 - Client
 - Server
- Greater burden placed on:
 - Server
- Multiple connections increase:
 - Server workload

Example 2.4

Figure 2.10 Example 2.4



Example 2.4 Illustration



- Separate TCP connection for each object
- Each connection requires setup and termination

Persistent Connections

- HTTP version 1.1 uses:
 - Persistent connections by default
- In persistent connection:
 - TCP connection remains open
- Same connection used for:
 - Multiple requests
 - Multiple responses
- Eliminates need for:
 - Repeated connection establishment

Operation of Persistent Connections

- Server keeps connection open:
 - After sending response
- Connection closed when:
 - Client requests closure
 - Time-out occurs
- Multiple objects can be retrieved:
 - Through single TCP connection

Data Length Information

- Sender usually includes:
 - Length of transmitted data
- Data length helps client:
 - Detect end of response
- Some situations prevent:
 - Knowing data length in advance

Unknown Data Length Situations

- Unknown data length occurs with:
 - Dynamic documents
 - Active documents
- Server informs client:
 - Length is unknown
- Server closes connection:
 - After sending complete data
- Connection closure indicates:
 - End of data transmission

Advantages of Persistent Connections

- Persistent connections save:
 - Time
 - Resources
- Only one set of:
 - Buffers
 - Variables
- Resource allocation needed:
 - Only once per connection
- Reduces:
 - Server overhead
 - Client overhead



Reduction in Round Trip Time

- Persistent connections avoid:
 - Repeated TCP handshakes
- Saves round trip time for:
 - Connection establishment
 - Connection termination
- Improves:
 - Communication efficiency
 - Web performance

Persistent Connection Illustration

Single TCP Connection

Client $\longleftrightarrow$ Server

Request 1 $\rightarrow$ Response 1

Request 2 $\rightarrow$ Response 2

Request 3 $\rightarrow$ Response 3

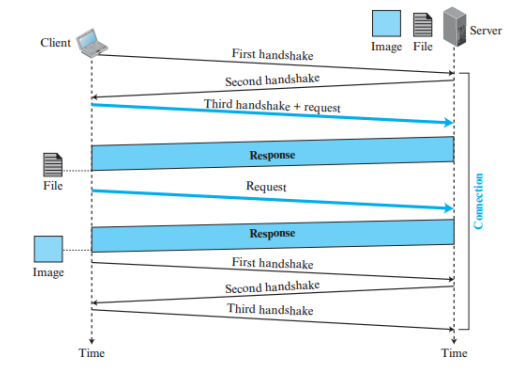
- One connection supports multiple transactions

Nonpersistent vs Persistent Connections

Feature	Nonpersistent	Persistent
TCP Connections	Multiple	Single
Setup Overhead	High	Low
Resource Usage	High	Lower
Round Trip Delay	More	Less
HTTP Version	Before 1.1	HTTP 1.1 Default

Example 2.5

Figure 2.11 Example 2.5



Example 2.5: Persistent Connection

- Same scenario as Example 2.4
- Client accesses:
 - Text file
 - Image file
- Both files located on:
 - Same server
- Persistent HTTP connection used:
 - Single TCP connection



Connection Handling in Persistent HTTP

- Only one:
 - Connection establishment
 - Connection termination
- Same TCP connection reused for:
 - Multiple object transfers
- Eliminates repeated:
 - TCP handshakes

Separate Requests in Persistent Connection

- Main text file retrieved:
 - Through first request
- Image file retrieved:
 - Through separate request
- Both requests use:
 - Same TCP connection
- Persistent connection keeps:
 - Communication channel open

Advantages in Example 2.5

- Reduced:
 - Connection setup overhead
 - Connection termination overhead
- Saves:
 - Round-trip time
 - System resources
- Requires:
 - Only one set of buffers
 - Only one set of variables
- Improves:
 - Communication efficiency



Example 2.5 Illustration

Single TCP Connection

Client $\longleftrightarrow$ Server

Request 1 $\rightarrow$ Retrieve Text File

Request 2 $\rightarrow$ Retrieve Image File

Single Connection Termination

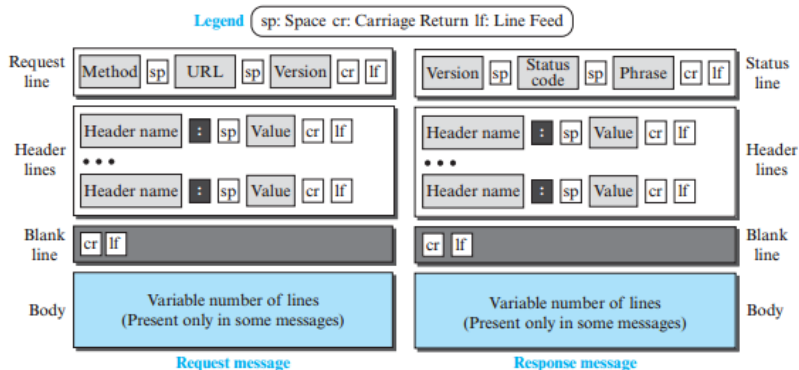
- Multiple requests handled using one TCP connection

Example 2.4 vs Example 2.5

Feature	Example 2.4	Example 2.5
Connection Type	Nonpersistent	Persistent
TCP Connections	Two	One
Connection Setup	Multiple	Single
Connection Termination	Multiple	Single
Efficiency	Lower	Higher
Overhead	Higher	Lower

Formats of the request and response messages

Figure 2.12 *Formats of the request and response messages*



HTTP Message Formats

- HTTP defines formats for:
 - Request messages
 - Response messages
- Request and response formats:
 - Similar in structure
- Each message contains:
 - Four sections
- Message formats standardized by:
 - HTTP protocol

HTTP Request Message Structure

- First section called:
 - Request Line
- Other sections include:
 - Header section
 - Blank line
 - Body section
- Request message sent by:
 - HTTP client



HTTP Response Message Structure

- First section called:
 - Status Line
- Other sections include:
 - Header section
 - Blank line
 - Body section
- Response message sent by:
 - HTTP server



Comparison of Request and Response Messages

- Request and response messages have:
 - Same section names
- However:
 - Contents of sections may differ
- Request message contains:
 - Client request information
- Response message contains:
 - Server response information

HTTP Message Format Illustration

Request Message	Response Message
Request Line	Status Line
Header Section	Header Section
Blank Line	Blank Line
Message Body	Message Body

- Structures are similar
- Contents differ according to message type

Table 2.1 *Methods*

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

Request Header Names

Table 2.2 *Request Header Names*

<i>Header</i>	<i>Description</i>
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later)
If-Modified-Since	If the file is modified since a specific date

HTTP Request Message

- First line of request message:
 - Request Line
- Request line contains:
 - Method
 - URL
 - Version
- Fields separated by:
 - Single space
- Line terminated by:
 - Carriage Return (CR)
 - Line Feed (LF)



Structure of Request Line

Method URL Version

- Method:
 - Type of request
- URL:
 - Address of web page
- Version:
 - HTTP protocol version

HTTP Methods

- HTTP version 1.1 defines:
 - Several request methods
- Common methods include:
 - GET
 - HEAD
 - PUT
 - POST
 - TRACE
 - DELETE
 - CONNECT
 - OPTIONS



GET and HEAD Methods

- GET method:
 - Most commonly used
 - Retrieves web page
- GET request usually contains:
 - Empty body
- HEAD method:
 - Retrieves only header information
- Used for:
 - Checking modification time
 - Testing URL validity
- HEAD response contains:
 - Header only
 - Empty body

PUT and POST Methods

- PUT method:
 - Opposite of GET
 - Uploads new web page to server
- PUT allowed only:
 - With permission
- POST method:
 - Sends information to server
- POST used for:
 - Adding information
 - Modifying web pages



Other HTTP Methods

- TRACE method:
 - Used for debugging
 - Server echoes request back
- DELETE method:
 - Deletes web page from server
- CONNECT method:
 - Reserved for proxy server use
- OPTIONS method:
 - Retrieves properties of web page

URL and Version Fields

- URL field defines:
 - Address and name of web page
- Version field specifies:
 - HTTP protocol version
- Current HTTP version:
 - HTTP/1.1

Request Header Lines

- Request message may contain:
 - Zero or more header lines
- Header lines provide:
 - Additional client information
- Example:
 - Requesting special document format
- Header line format:
 - Header Name : Value

Format of Header Lines

Header-Name : Header-Value



- Header name:
 - Identifies information type
- Header value:
 - Associated parameter value

Body of Request Message

- Request body may be:
 - Present or absent
- Body commonly used with:
 - PUT method
 - POST method
- Body may contain:
 - Comments
 - Uploaded files
 - Data for web page modification

Summary of HTTP Methods

Method	Purpose
GET	Retrieve web page
HEAD	Retrieve header only
PUT	Upload new page
POST	Send/modify data
TRACE	Debugging
DELETE	Delete web page
CONNECT	Proxy communication
OPTIONS	Request page properties

Response Header Names

Table 2.3 *Response Header Names*

<i>Header</i>	<i>Description</i>
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

HTTP Response Message

- Response message consists of:
 - Status line
 - Header lines
 - Blank line
 - Body
- Response message sent by:
 - HTTP server
- Body may be:
 - Present or absent



Status Line in Response Message

- First line called:
 - Status Line
- Status line contains:
 - Version
 - Status Code
 - Status Phrase
- Fields separated by:
 - Spaces
- Line terminated by:
 - Carriage Return (CR)
 - Line Feed (LF)



Structure of Status Line

Version Status-Code Status-Phrase

- Version:
 - HTTP protocol version
- Status Code:
 - Numeric request status
- Status Phrase:
 - Text explanation of status



HTTP Version Field

- First field specifies:
 - HTTP protocol version
- Current version: 
 - HTTP/1.1
- Helps client identify:
 - Supported protocol features

HTTP Status Codes

- Status code contains:
 - Three digits
- Indicates:
 - Result of client request
- Status codes grouped into:
 - 100 range
 - 200 range
 - 300 range
 - 400 range
 - 500 range

Meaning of Status Code Ranges

- 100 range:
 - Informational messages
- 200 range:
 - Successful request
- 300 range:
 - Redirection to another URL
- 400 range:
 - Client-side errors
- 500 range:
 - Server-side errors

Status Phrase

- Status phrase:
 - Text explanation of status code
- Helps user understand:
 - Meaning of response
- Example:
 - 200 OK
 - 404 Not Found

Response Header Lines

- Response may contain:
 - Zero or more header lines
- Header lines provide:
 - Additional information
- Information sent from:
 - Server to client
- Example:
 - Information about document

Format of Response Header Lines

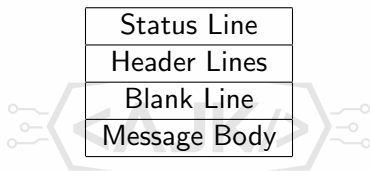
Header-Name : Header-Value

- Header name:
 - Defines information type
- Header value:
 - Defines associated value

Body of Response Message

- Body contains:
 - Requested document
- Document transferred from:
 - Server to client
- Body absent when:
 - Response is an error message
- Browser uses body to:
 - Display web content

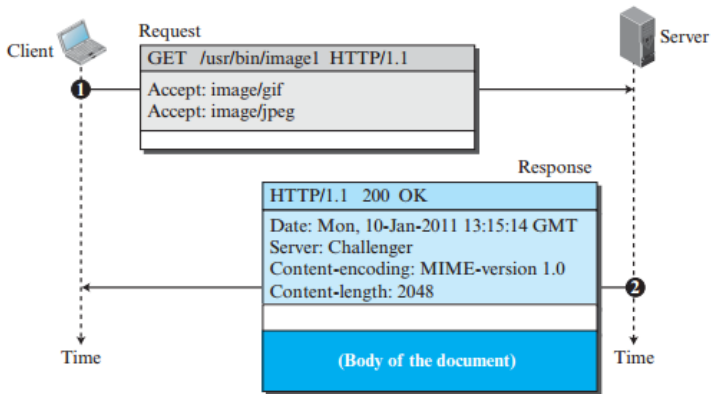
HTTP Response Message Structure



- Structure used in all HTTP responses

Example 2.6

Figure 2.13 Example 2.6



Example 2.6: Retrieving a Document

- Example demonstrates:
 - HTTP request and response
- Client retrieves:
 - Image document
- HTTP method used:
 - GET
- Requested path:
 - /usr/bin/image1

HTTP Request Line in Example 2.6

- Request line contains:

- Method
- URL
- HTTP version

- Method:

- GET

- URL:

- /usr/bin/image1

- Version:

- HTTP/1.1



Request Header Information

- Request header contains:
 - Two header lines
- Header lines indicate:
 - Acceptable image formats
- Supported formats:
 - GIF
 - JPEG
- Request body:
 - Not present

HTTP Response Message in Example 2.6

- Response message contains:
 - Status line
 - Four header lines
 - Body
- Server sends:
 - Requested image document



Response Header Information

- Response headers provide:
 - Date information
 - Server information
 - Content encoding
 - Document length
- Content encoding specifies:
 - MIME version
- Document length defines:
 - Size of transmitted data

Body of the Response

- Body follows:
 - Response headers
- Body contains:
 - Requested image document
- Browser uses body to:
 - Display retrieved content

Illustration of Example 2.6

Request Message
GET /usr/bin/image1 HTTP/1.1 Accept: GIF Accept: JPEG
No Body

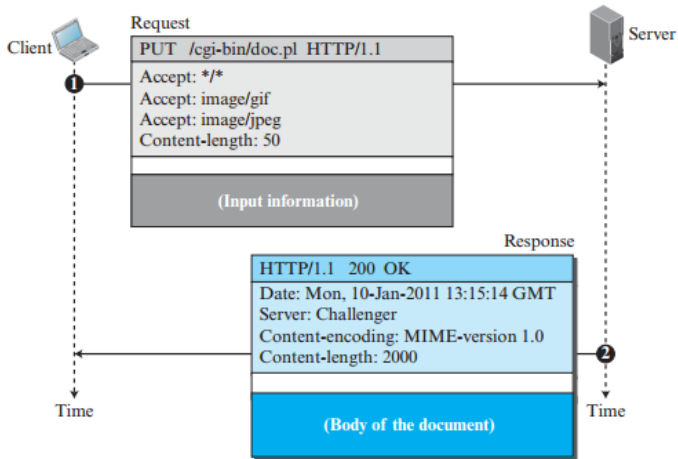
Response Message
HTTP/1.1 200 OK Date Information Server Information MIME Encoding Content Length
Image Document Body

Key Points from Example 2.6

- GET method used to:
 - Retrieve resource
- Request specifies:
 - Accepted document formats
- Response provides:
 - Status information
 - Document details
 - Requested content
- MIME encoding helps:
 - Identify document format

Example 2.7

Figure 2.14 Example 2.7



Example 2.7: Uploading a Web Page

- Client wants to:
 - Send a web page to server
- HTTP method used:
 - PUT
- Purpose of PUT:
 - Post new web page on server



Request Line in Example 2.7

- Request line contains:
 - Method
 - URL
 - HTTP version
- Method:
 - PUT
- Version:
 - HTTP/1.1
- URL specifies:
 - Destination web page location



Request Headers and Body

- Request message contains:
 - Four header lines
- Headers provide:
 - Additional request information
- Request body contains:
 - Web page to be uploaded
- PUT request transfers:
 - Entire document to server

Response Message in Example 2.7

- Server sends:
 - Response message
- Response contains:
 - Status line
 - Four header lines
 - Body
- Status line indicates:
 - Result of upload operation



CGI Document in Response

- Response body contains:
 - Created CGI document
- CGI stands for:
 - Common Gateway Interface
- CGI document generated by:
 - Server-side processing
- Response confirms:
 - Successful document creation

Illustration of Example 2.7

Request Message
PUT URL HTTP/1.1
Header Line 1
Header Line 2
Header Line 3
Header Line 4
Web Page Body

Response Message
HTTP/1.1 Status Code
Header Line 1
Header Line 2
Header Line 3
Header Line 4
CGI Document Body

Key Points from Example 2.7

- PUT method used for:
 - Uploading documents
- Request body carries:
 - Actual web page content
- Server processes uploaded page:
 - Creates CGI response
- Response confirms:
 - Status of upload operation

Conditional Request

- HTTP client can add:
 - Conditions in request message
- Server checks:
 - Whether condition is satisfied
- If condition is met:
 - Requested web page sent
- Otherwise:
 - Server informs client

Purpose of Conditional Requests

- Conditional requests help:
 - Avoid unnecessary data transfer
- Client requests document only when:
 - Specific condition holds true
- Improves:
 - Network efficiency
 - Communication performance

If-Modified-Since Header

- Common conditional header:
 - If-Modified-Since
- Client specifies:
 - Date and time
- Server sends page only if:
 - Page modified after specified time
- Used to check:
 - Updated version of document

Working of If-Modified-Since

- Client sends request with:
 - If-Modified-Since header
- Server compares:
 - Modification time of web page
- If page updated:
 - Server sends latest page
- If page unchanged:
 - Server avoids sending duplicate data

Example of Conditional Request

If-Modified-Since: 10 Jan 2026 10:00:00

- Server sends page only if:
 - Modified after specified date and time
- Otherwise:
 - No updated page transferred

Advantages of Conditional Requests

- Reduces:
 - Network traffic
 - Unnecessary retransmission
- Saves:
 - Bandwidth
 - Time
- Improves:
 - Web efficiency
 - Response speed
- Useful for:
 - Frequently accessed web pages



Example 2.8: Conditional Request

- Client sends:
 - Conditional HTTP request
- Condition based on:
 - Modification date and time
- Request uses:
 - If-Modified-Since header
- Goal:
 - Retrieve page only if updated

HTTP Request in Example 2.8

```
GET http://www.commonServer.com/information/file1 HTTP/1.1
If-Modified-Since: Thu, Sept 04 00:00:00 GMT
```

- 
- GET method used to:
 - Retrieve web page
 - Header specifies:
 - Required modification condition

Meaning of the Condition

- Client requests page only if:
 - Modified after Sept 04
- Server checks:
 - Last modification time of file
- If file unchanged:
 - Server avoids sending document

HTTP Response in Example 2.8

HTTP/1.1 304 Not Modified Date: Sat, Sept 06 08:16:22 GMT Server: commonServer.com
(Empty Body)

Status Code 304 Not Modified

- Response status code:
 - 304 Not Modified
- Indicates:
 - Requested file not updated
- Server determines:
 - File unchanged after specified date
- No document retransmission required

Response Headers in Example 2.8

- Response contains:
 - Date header
 - Server header
- Date header specifies:
 - Time of response generation
- Server header specifies:
 - Name of responding server

Empty Response Body

- Response body is:
 - Empty
- Reason:
 - File not modified
- Sending document again would:
 - Waste bandwidth
- Client can continue using:
 - Cached copy of file

Advantages of Example 2.8

- Reduces:
 - Network traffic
 - Duplicate transmissions
- Saves:
 - Bandwidth
 - Response time
- Improves:
 - Web efficiency
 - Cache utilization

